

A client instantiates a `DisplayMoney` object like this:

---

```
oDisplayMoney = DisplayMoney(widow, (100, 100), 1234.56)
```

---

With this line, the `__init__()` method in `DisplayMoney` ❷ will run and override the `__init__()` method in the base class. This method does some initialization, including saving any client preferences for the currency symbol, the side on which to show the symbol, and whether or not we should show cents, all in instance variables ❸. The method ends with a call to the `__init__()` method of the base class, `DisplayText` ❹ (which it finds by calling `super()`), and passes on the data required by that method.

Later, the client makes a call like this to show a value:

---

```
oDisplayMoney.setValue(12233.44)
```

---

The `setValue()` method ❺ in the `DisplayMoney` class runs to create a version of the amount of money formatted as a currency value. The method ends by calling the inherited `setValue()` method in the `DisplayText` class ❻ to set the new text to display.

When a call is made to any other method with an instance of `DisplayMoney`, the version residing in `DisplayText` will run. Most importantly, every time through the loop, the client code should call `oDisplayMoney.draw()`, which draws the field in the window. Since `DisplayMoney` does *not* have a `draw()` method, that call will go to the `DisplayText` base class, which does have a `draw()` method.

## ***Example Usage***

[Figure 10-5](#) shows the output of an example program that takes advantage of both the `InputNumber` and `DisplayMoney` classes. The user enters a number into an `InputNumber` field. When the user presses `OK` or `ENTER`, that value will be displayed in the two `DisplayMoney` fields. The first field shows the number with decimal places, and the second rounds to the closest dollar using different initial settings.

[Listing 10-5](#) contains the full code of the main program. Notice that the code creates a single `InputNumber` object and two `DisplayMoney` objects.